CSS

Css stands for cascading style sheets

CSS is the language we use to style an HTML document

Anatomy of css structure

Selector

```
p {

    color: blue;

}
```

Each selector wrapped in a curly braces

Property

Property Value

Declaration

Three ways to bind CSS with HTML

- ❑ Inline Styling
- ❑ Internal Styling
- ❑ External Styling

Inline Styling

Inline styling is a technique in HTML that allows you to style directly to an HTML element using the style attribute

```
<h1 style="color: blue"> School4u </h1>
```

Internal Styling

Internal styling applies CSS within the <style> tag in an HTML document's <head>, affecting the whole page.

```
<head>
    <style>

        h1 {
            color: blue;
        }

    </style>
</head>
```

External Styling

External styling in CSS involves linking an HTML document to a separate CSS file using the <link> tag.

```
<head>

    <link rel="stylesheet" href="styles.css">

</head>
```

Cascading Order

- Inline > Internal > External
- Keep the concept of override in mind while styling

CSS Comments

- Comments are ignored by browsers.
- It is only for your reference.

```
p{
    background-color: red;
    /* font-size: 80px; */
    color: white;
    padding: 20px; /* create space inside the box*/
}
```

Selectors in CSS

- Universal Selector (*)
- Element Selector (h1, p, div, etc)
- Id Selector (#)
- Class Selector (.)
- Group Selector (h1, .box, #wick)
- Descendant Selector (e.g., div p)
- Direct Child Selector (>)

NOTE:

- Descendant Selector matches all elements that are descendants of a specified element..
- Direct Child Selector is used to select elements with a specific parent. Elements that are not directly a child of the specified parent, are not selected.

Colors in CSS

- Used to set the color of text, background, border, etc.
- Colors are specified using predefined color names, or RGB, HEX, HSL, RGBA, HSLA values.

```
background-color: rgb(255, 99, 71); /* RGB */
background-color: #ff6347; /* HEX */
background-color: (9, 100%, 64%); /* HSL */
background-color: tomato; /* css color name */
```

```
background-color: rgb(255, 99, 71); /* RGBA */
background-color: (9, 100%, 64%); /* HSLA */
```

- Avoid all the above codes and use “Color Zilla” Extension of Chrome and color-picker of css to pic color

Text & Background Color

- You can set the color of text using "color" property.
- You can set the Background Color of Element using "background-color" property.

```
p {
    color : red;
    background-color: black;
}
```

Border In CSS

- The CSS border properties allow you to specify the style, width, and color of an element's border.

```
p {
    border-style: solid; /* required*/
    border-width: 3px;
    border-color: red;
}
```

NOTE: border-style property is required to show border.
Note: If border-color is not set, it inherits the color of the element.

Shorthand Property

The CSS specification defines shorthand properties to group the definition of common properties acting on the same theme.

For Example :
The border property is a shorthand property for the following individual border properties:
border-width
border-style (required)
border-color

```
p {
  border: 5px solid red;
}
```

Let’s Understand Some other shorthand strategy with one more example:

For Example :

There are four sides of border and we can style all four sides in different ways:

```
.SpecifyEachSides {
  border-top-color: red;
  border-right-color: blue;
  border-bottom-color: green;
  border-left-color: black;
}
.AllFourSidesInSingleValue {
  border-color: red;
}
.TopBottom-LeftRight {
  border-color: red blue;
}
.Top-RightLeft-Bottom {
    border-style: solid;
  border-color: red blue green;
}
.Top-Right-Bottom-Left {
  border-color: red green blue black;
}
```

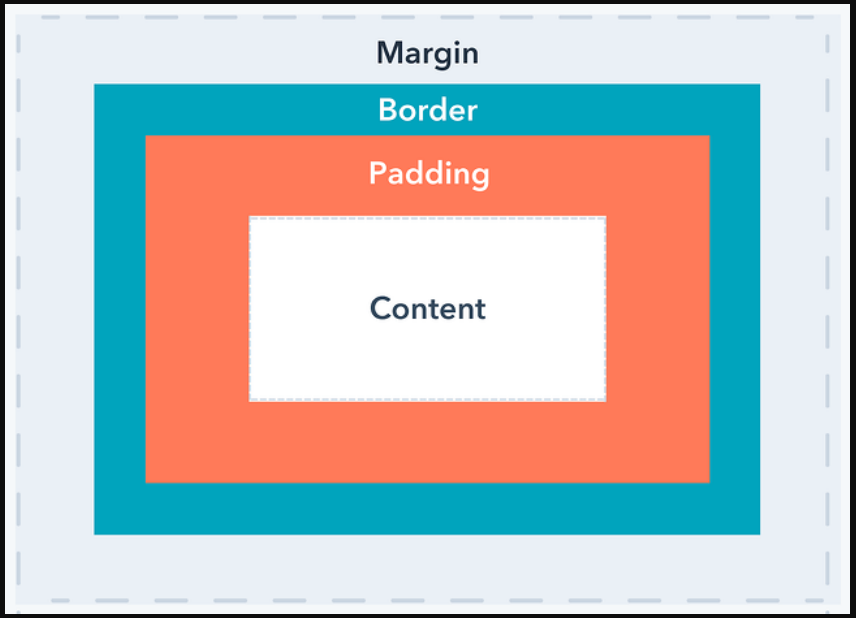
NOTE: The above strategy also works with border-width and border-style.

Margin v/s Padding in CSS

- margin is the space around an element's border, while padding is the space between an element's border and the element's content

```
p {
  margin-top: 100px;
  margin-right: 150px;
  margin-bottom: 200px;
  margin-left: 80px;
}
```

```
p {
  padding-top: 100px;
  padding-right: 150px;
  padding-bottom: 200px;
  padding-left: 80px;
}
```



- To shorten the code, it is possible to specify all the margin/padding properties in one property

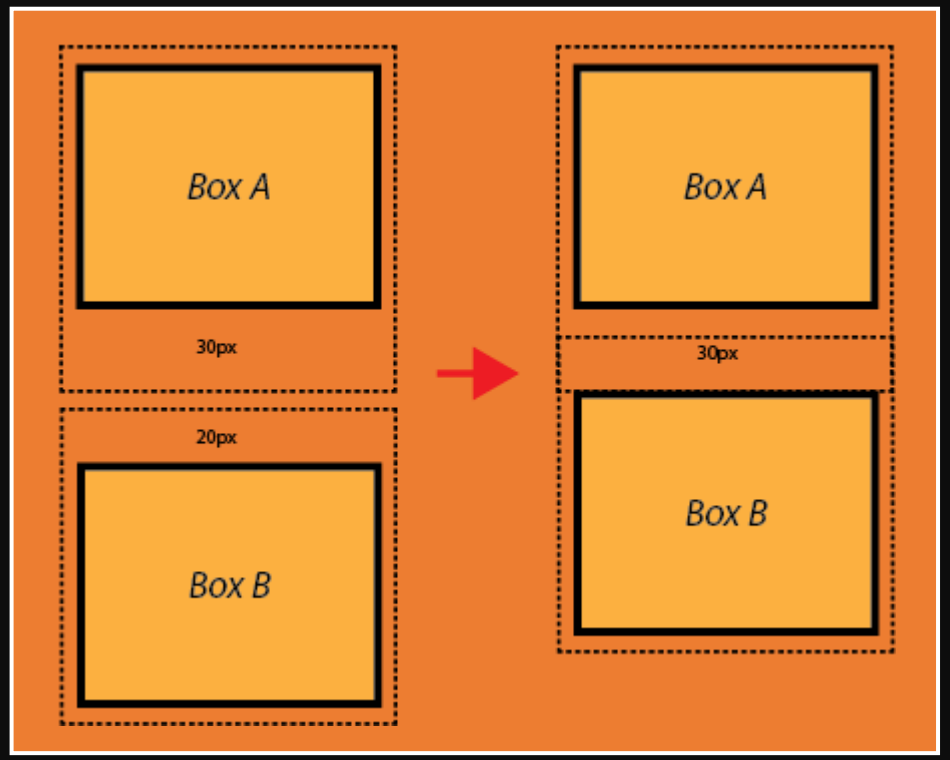
```
p {
    margin: 100px 150px 200px 80px;
    padding: 100px 150px 200px 80px;
}
```

NOTE:
- Negative values are allowed for margin.
- Negative values are not allowed for padding.

Margin Collapse

- Sometimes two margins collapse into a single margin.
- Top and bottom margins of elements are sometimes collapsed into a single margin that is equal to the largest of the two margins.
- This does not happen on left and right margins! Only top and bottom margins!

```
.box1 {
  margin: 0 0 50px 0;
}
.box2 {
  margin: 20px 0 0 0;
}
```



Width & Height in CSS

- The height and width properties are used to set the height and width of an element.

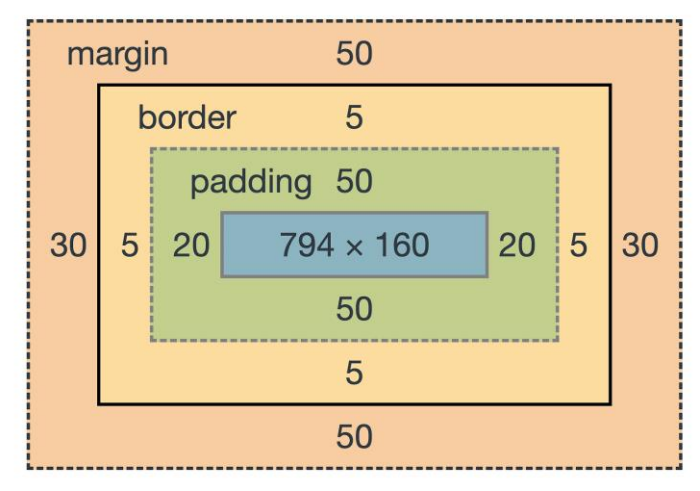
```
div {
    height: 200px;
    width: 50%;
    background-color: lightseagreen;
}
```

Max and Min Width/Height

```
div {
    width: 100%;
    max-width: 500px;
    padding: 20px;
}
```

CSS Box Model

- All HTML elements can be considered as boxes.
- In CSS, the term "box model" is used when talking about design and layout.



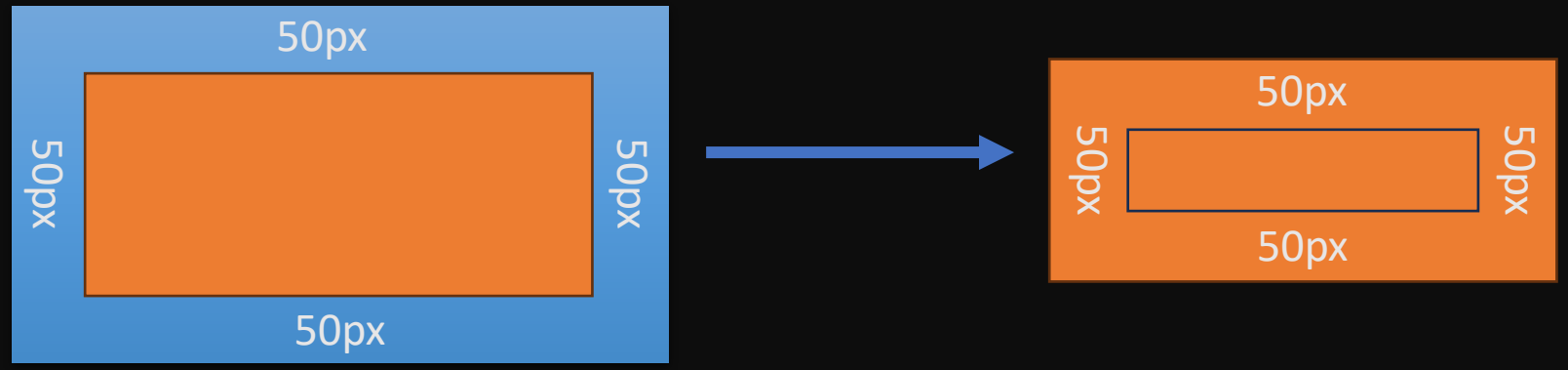
NOTE:
- margin is not included in the actual size of the box.
- The box's total width and height stops at the border

box-sizing: border-box

- The CSS box-sizing property allows us to include the padding and border in an element's total width and height.

```
.box{
    width: 300px;
    height: 50px;
    padding: 50px;
    background-color: red;
}
```

```
.box{
    width: 300px;
    height: 50px;
    padding: 50px;
    background-color: red;
    box-sizing: border-box;
}
```



CSS TEXT & FONT

font-size

font-weight

text-align

direction

text-transform

letter-spacing

line-height

word-spacing

white-space

Font Family

The font-family property in CSS specifies the typeface that should be used for text within an element.

Common Generic Font Families

- Arial (default)
- Serif
- Sans-serif
- Monospace

Position

- **Static** (By Default)
- **Relative** (positioned relative to its normal position)
- **Absolute** (positioned relative to the nearest positioned ancestor)
- **fixed** (positioned relative to the viewport, which means it always stays in the same place even if the page is scrolled)
- **Sticky** (positioned based on the user's scroll position)

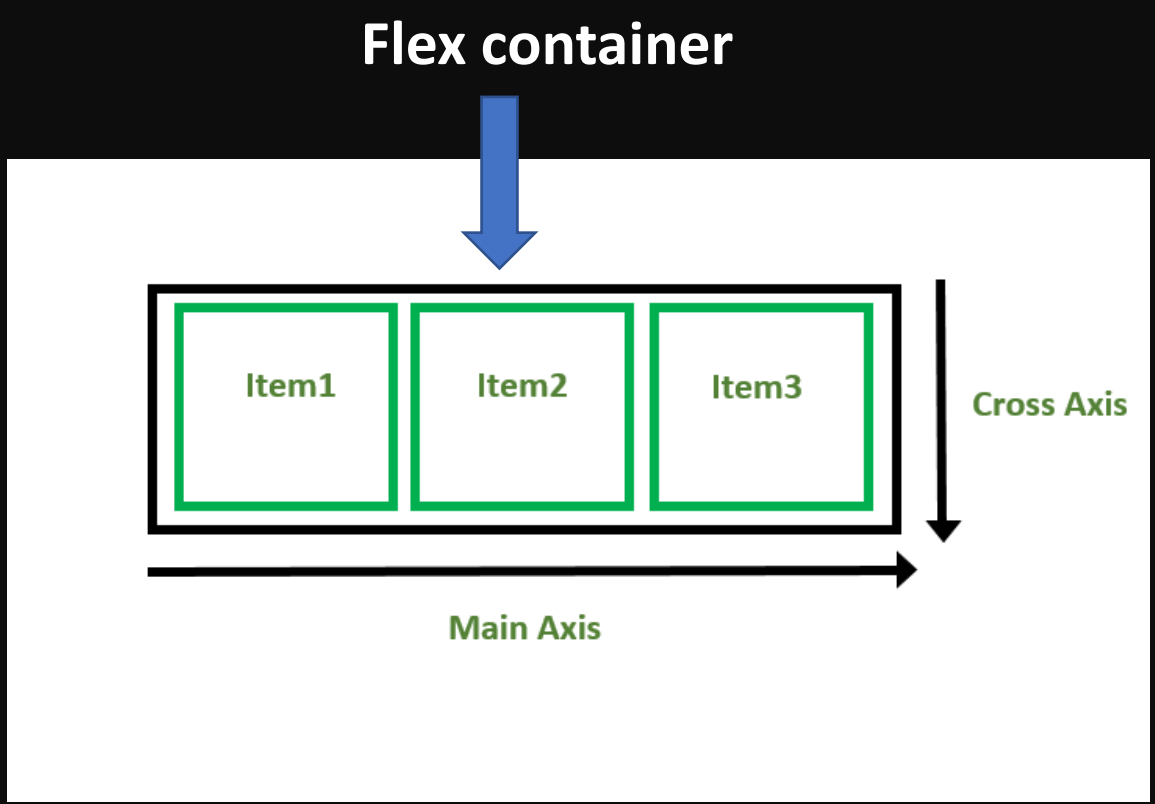
NOTE:

- Static positioned elements are not affected by the top, bottom, left, and right properties.
- If there are no positioned ancestors, absolute uses the document body as its reference point
- if two elements are absolutely positioned without a specified z-index, the one that is placed later in the HTML code will appear on top. If z-index values are specified, the element with the higher z-index will be on top, regardless of their order in the document.

Flex Box

- To start using the Flexbox model, you need to first define a flex container.
- To define a flex container you need to set display property as flex.

```
.container {
    display: flex;
  }
```



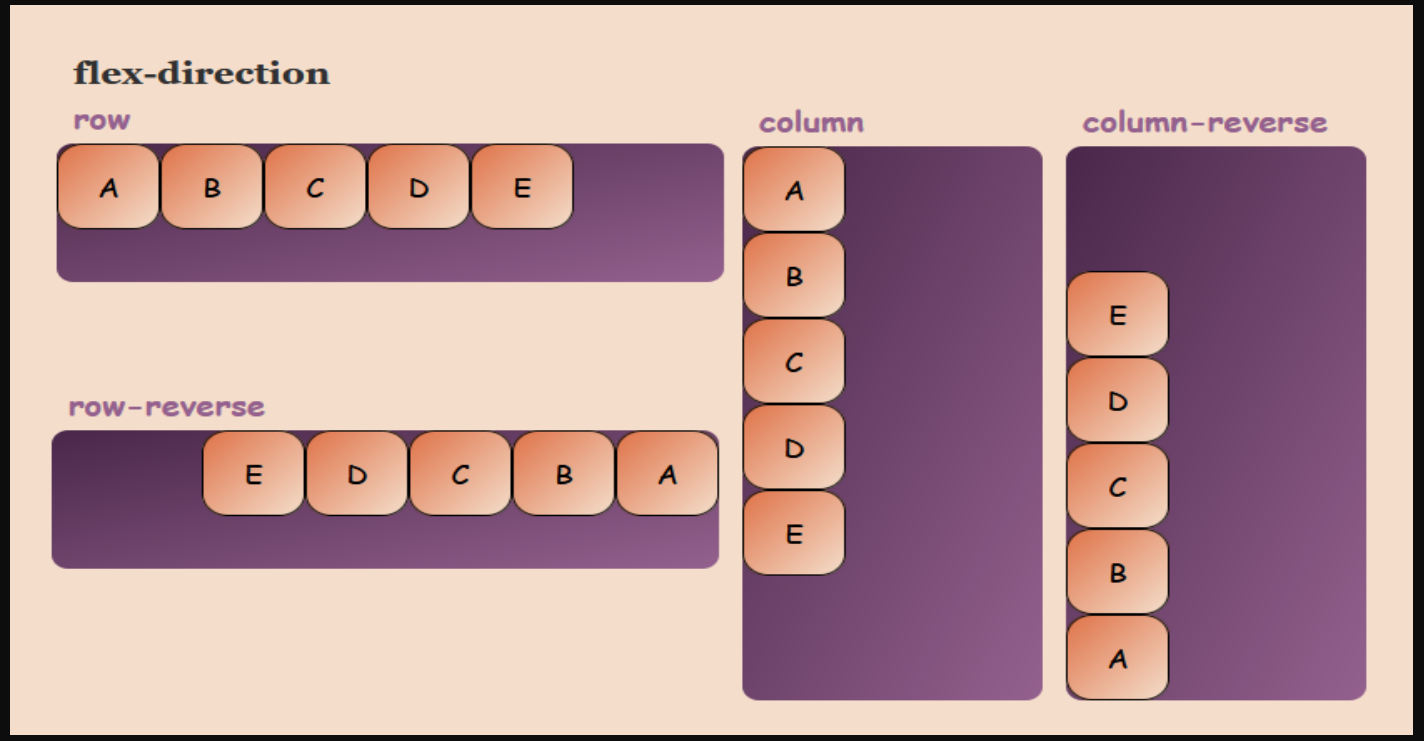
Flexbox Container Properties

- flex-direction
- flex-wrap
- justify-content
- align-items
- align-content

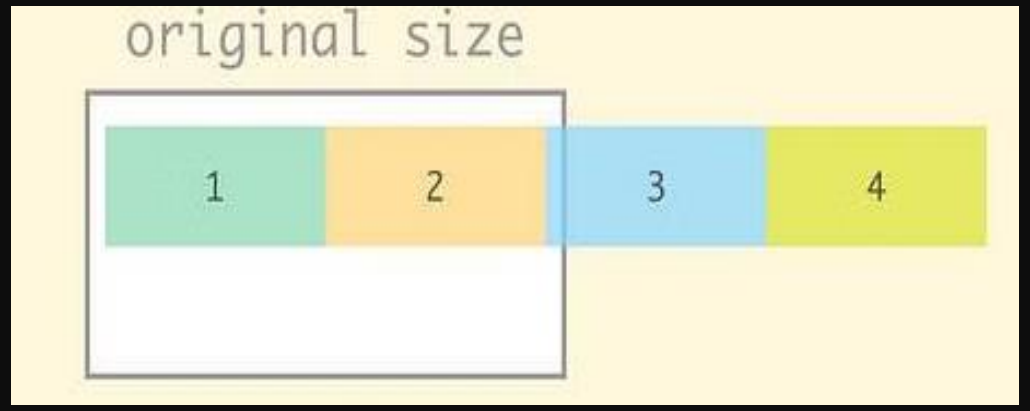
flex-direction

Their are four Flex Directions :-

- ❑ **row**
- ❑ **row-reverse**
- ❑ **column**
- ❑ **column-reverse**

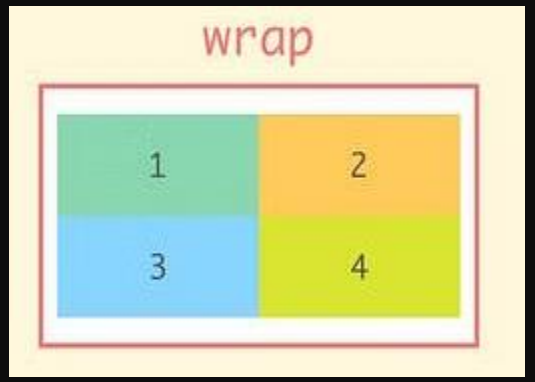


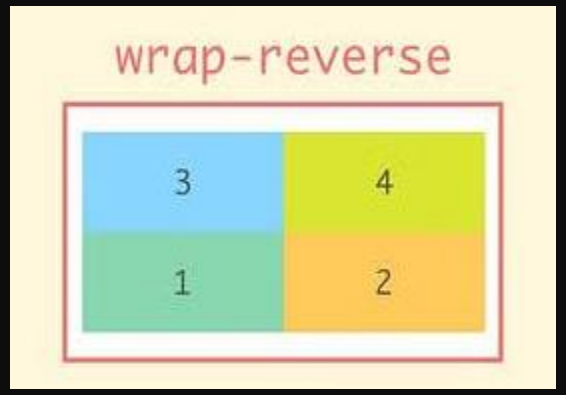
flex-wrap

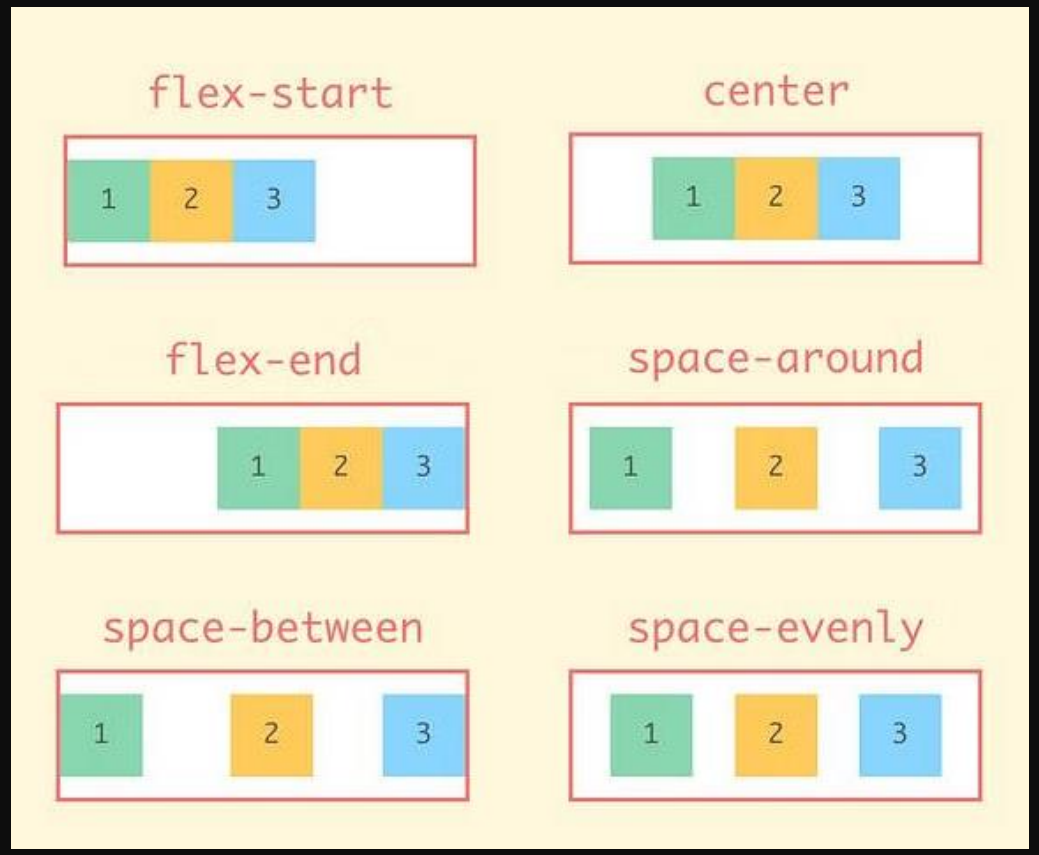


Their are three choices of flex-wrap :

- ❑ **wrap**
- ❑ **nowrap**
- ❑ **wrap-reverse**



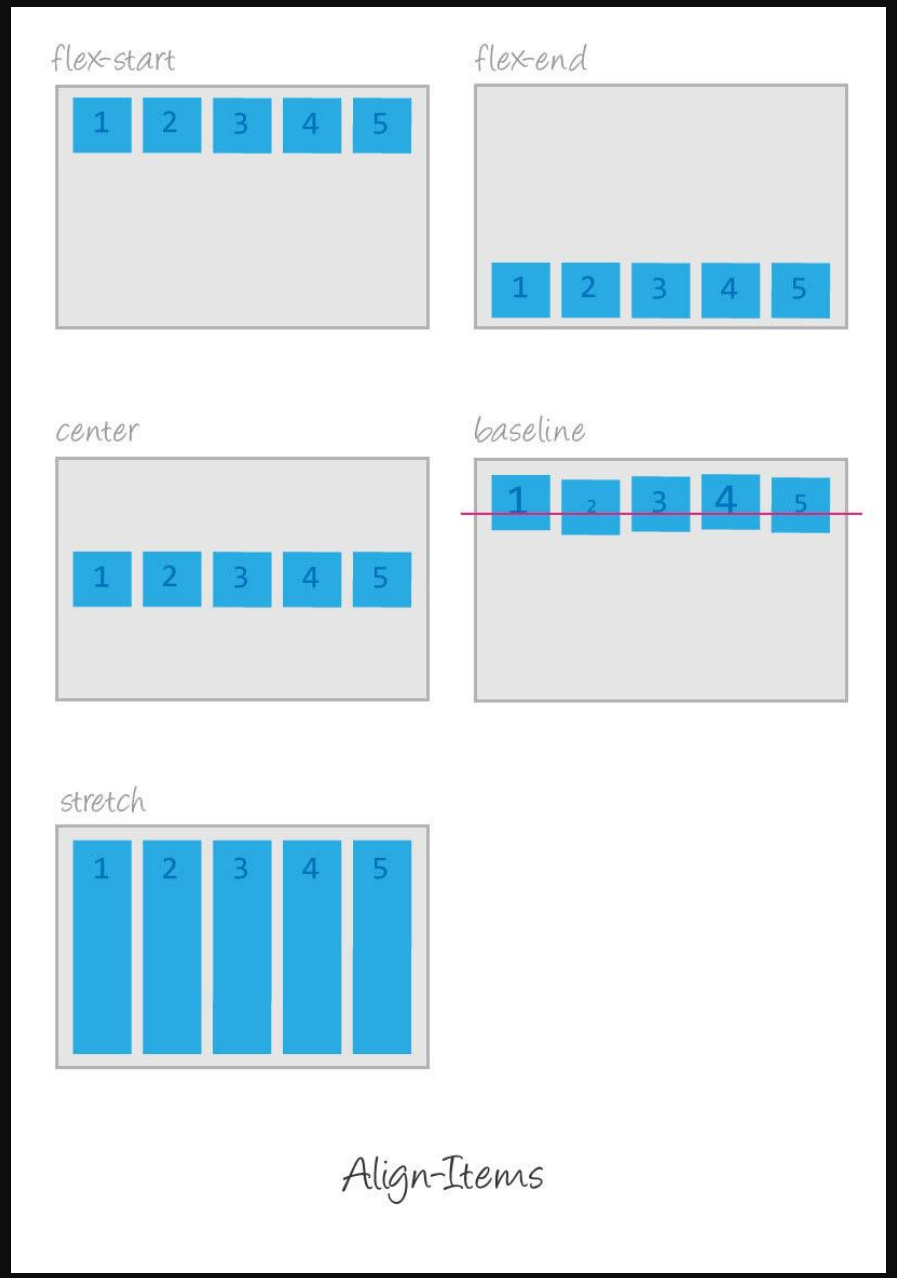




justify-content

Their are five choices for justify-content :

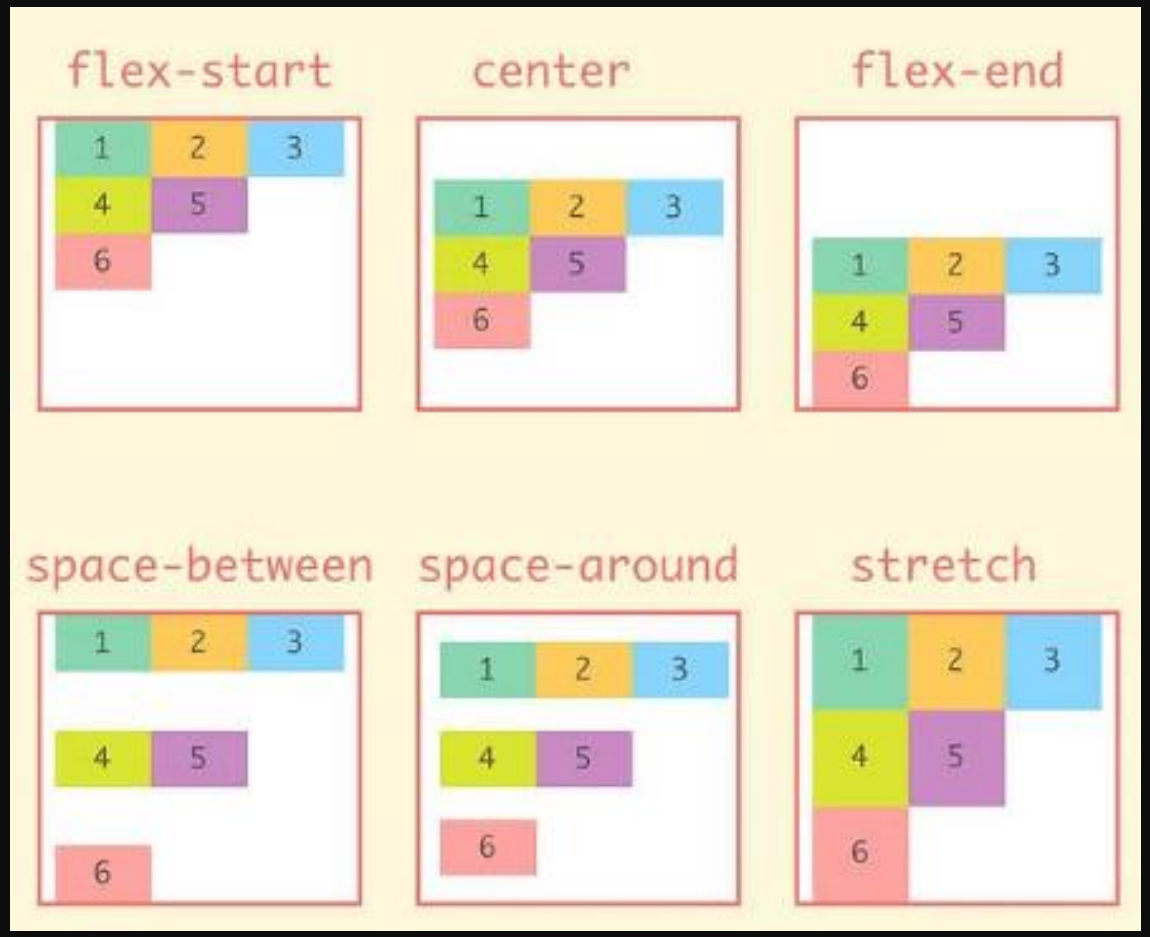
- ❑ **flex-start**
- ❑ **center**
- ❑ **flex-end**
- ❑ **space-around**
- ❑ **space-between**
- ❑ **space-evenly**



align-items

Their are five choices to align-items :

- ❑ **flex-start**
- ❑ **flex-end**
- ❑ **center**
- ❑ **stretch**
- ❑ **baseline**



align-content

Their are six choices to align-content :

- ❑ **flex-start** ,
- ❑ **center**
- ❑ **flex-end**
- ❑ **space-between**
- ❑ **space-around**
- ❑ **stretch**

Strategy to center the elements using flex

Make parent display as **flex** and set both the **justify-content** and **align-items** properties to **center**.

```
.flex-container {
  display: flex;
  height: 300px;
  justify-content: center;
  align-items: center;
}
```

Units in CSS

There are two main types of units: **absolute units** and **relative units**.

Absolute Units

Absolute units are fixed and do not change based on other factors, such as the size of the viewport or font size.

Generally, we use **px (pixels**)

Relative Units

Relative units are based on other values, such as the font size of the parent element or the viewport size.

Unit	Description	Value Example
%	Percentage of the parent element’s size, enabling responsive design.	50% (50% of parent width)
em	Relative to the font size of the element itself; 2em is twice the current font size.	1em = 16px (if the font size is 16px)
rem	Relative to the root element's font size; 1rem equals the root font size.	1rem = 16px (if root font size is 16px)
vw	1% of the viewport width; 50vw is half the viewport width.	1vw = 1% of viewport width
vh	1% of the viewport height; 100vh is the full height of the viewport.	1vh = 1% of viewport height